

Rusting Away (or: packing the entire Crossref database into a SQLite file)

Cite as: Martin Paul Eve, "Rusting Away (or: packing the entire Crossref database into a SQLite file)", *eve.gd: Martin Paul Eve*, January 09, 2024, <https://doi.org/10.59348/5tkbp-dpa74>.



Over the past few weeks I've been working to pack the entire Crossref database into a distributable SQLite file. While this sounds somewhat insane – the resulting file is 900GB – it's quite a cool project for, say, embedded systems work in situations where no internet connection is available. It also provides speedy local indexed lookups, working faster than the internet-dependent API ever could.

There were some snags I hit along the way:

1. I had to write the whole thing in Rust. It turns out that you can do it in Python, but I got much better speeds doing it all in Rust.
2. An ORM approach in Python was far too slow.
3. Setting the PRIMARY KEY to be the DOI on the database was a bad idea. There are so many commonalities in prefixes that the B-tree just grew and grew until the whole program was abort(3)-ed for an out of memory error, taking down the terminal emulator in which it was running, too. Rust does *not* recover well from OOM errors.
4. It's best to build the index of DOIs at the end, once the data is in place.

5. In Rust, you can use a bounded channel to read data and block until it's been processed in a writer thread. This was a neat way of reading the tar.gz files at max speed but not exceeding the memory capacity of the host machine,